



**Department of Electrical, Computer
& Biomedical Engineering**
Faculty of Engineering & Architectural Science

Program: Computer Engineering

Course Number	COE 892
Course Title	Distributed Cloud Computing
Semester/Year	Winter 2026
Instructor	Muhammad Jaseemuddin

Report Title	Project Proposal
--------------	------------------

Section No.	09
Submission Date	February 3, 2026
Due Date	February 6, 2026

Name	Student ID	Signature*
Krish Patel	xxxx91722	K.P

(Individual Project; approval Pending)

(Note: remove the first 4 digits from your student ID)

*By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a “0” on the work, an “F” in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at:
<https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

COE892 Project Proposal

ProxSyncQ: Distributed VM File Sync + Resilient Job Queue on a ProxMox Private Cloud

Introduction + Problem Statement

Modern teams routinely run multi-VM and multi-service systems where both data and computation must remain available even when a node fails, a network segment becomes unstable, or workloads spike. In practical private-cloud environments, two recurring infrastructure problems appear together:

1. **Keeping critical files and configuration synchronized across multiple nodes** so services can be redeployed or recovered quickly if one node goes down.
2. **Distributing background work reliably across nodes** so the system continues processing jobs even if workers crash or nodes restart.

This project addresses both problems by building a small private-cloud platform that combines:

- A distributed VM file synchronization subsystem with conflict detection and recovery behaviors.
- A distributed job queue platform that supports at-least-once processing with idempotency and deduplication (and explores exactly once design constraints as a stretch goal).

The combined system is designed for an industrial private cloud setting where high availability, repeatability, and operational clarity matter as much as application features.

Proposed Solution

The project will implement an integrated infrastructure stack called **ProxSyncQ**, deployed across three equal-capacity VMs. Each VM runs:

- A **Sync Agent** responsible for file monitoring, change packaging, and applying remote updates.
- A **Worker Agent** that processes jobs from a queue with retries and crash recovery.
- A minimal set of **control-plane services** (logically centralized but made fault-aware) for metadata, deduplication, and observability.

The key idea is that file synchronization and job execution reinforce each other:

- File changes generate sync events that become queued jobs, allowing controlled replication and backpressure handling.
- Jobs can be scheduled on any healthy VM because required inputs are either replicated locally or available via a shared storage option, depending on the evaluated storage strategy.

System architecture and components

The system runs on **three identical ProxMox VMs** (4 vCPU, 8 GB RAM, 100 GB disk each). Every VM participates in both subsystems so the platform can continue operating when one node is degraded or temporarily unavailable. Control-plane services start on one VM for simplicity, but the design includes restart and recovery procedures so the demo can show resilience.

File sync subsystem

Goal: keep a chosen directory set (configs, artifacts, job inputs/outputs) synchronized across VMs under failures, partitions, and concurrent edits.

Each VM runs a **Sync Agent** that monitors the directory using filesystem events and periodic scans. Changes are converted into chunked updates with hashes so replication can be incremental and verifiable. Updates are applied using an atomic staging approach to avoid partial writes.

- **Sync Agent (per VM):** watcher + periodic scan, chunk hash, send/apply, atomic write (stage then swap)
- **Metadata:** version vectors or logical clocks to detect concurrent writes and out-of-order arrival
- **Conflict handling:** detect concurrency, preserve both versions, mark conflict state, push into a “manual merge” queue
- **Recovery:** offline nodes replay missed updates on return; periodic audit jobs verify convergence and repair missing chunks

Job queue subsystem

Goal: distribute background work across the three VMs reliably, even if a worker crashes mid-task.

The queue subsystem provides **durable job submission**, worker leasing with heartbeats, retries, and deduplication. The base guarantee is **at-least-once execution**, made safe using idempotency keys and a dedupe table so a retried job does not corrupt results.

- **Job API:** submit sync jobs, audit jobs, and synthetic test jobs with payload + priority
- **Broker:** durable queue to retain jobs and enable redelivery
- **Workers (per VM):** pull jobs, acquire lease, heartbeat, acknowledge on success
- **Result store + dedupe:** job state, attempts, idempotency keys, completion markers

Exactly-once is treated as a design discussion and a stretch goal for specific job types where transactional patterns make it realistic, but the implementation focus remains robust at-least-once semantics.

Storage strategy comparison

Two storage models will be implemented and compared with measurements before selecting the final approach.

In **replication-first**, each VM stores a local replica of the synchronized data, maximizing availability when a node or storage target is down. In **NAS-style shared storage**, VMs read/write shared files through a central storage service, simplifying consistency but introducing a shared dependency. The final decision will be justified using measured convergence time, bandwidth use, and recovery behavior, framed in how industry chooses between shared storage simplicity and replicated resilience.

- A) **Replication-first:** local replicas on all VMs
 - Pros: survives shared-storage outages, local reads, stronger “edge/private cloud” story
 - Cons: replication traffic, conflicts to manage
- B) **NAS-style shared storage + selective replication:**
 - Pros: simpler consistency, less replication overhead
 - Cons: central dependency, network/storage bottlenecks, availability risk

Timeline aligned to deliverables

The timeline is structured to ensure meaningful progress by the interim report and a polished, demo-ready system by the final deadline.

- **By Feb 6:** finalize proposal, architecture, initial implementation plan
- **Feb 7 to Feb 23:** MVP: broker + workers + basic sync pipeline + initial observability
- **Feb 24 to Mar 8:** integrate sync-as-jobs, conflict detection MVP, failure scripts, early metrics
- **Mar 9:** interim report: design + partial implementation + early results
- **Mar 10 to Mar 23:** storage comparison (replication vs NAS) + robustness improvements
- **Mar 24 to Mar 30:** finalize README, repeatable demo script, final measurement runs, final report + code submission
- **Mar 30 to Apr 10:** TA demo window (fault injection + recovery + metrics)

Contribution matrix

This project is proposed to be completed individually.

- **Krish Patel:** 100% (design, ProxMox setup, implementation, testing, evaluation, demo, writing)

Sources + Git Repo

- **Server Portfolio + Experience** - <https://server.krishadmin.com>
- **GitHub Repo** - <https://github.com/KrishPAdmin/ProxSynQ>
- https://pve.proxmox.com/wiki/Cloud-Init_Support
- https://pve.proxmox.com/wiki/Proxmox_VE_API
- <https://www.rabbitmq.com/docs/reliability>
- <https://pve.proxmox.com/pve-docs/chapter-pvesm.html>
- <https://groups.csail.mit.edu/tds/papers/Gilbert/Brewer2.pdf>
- <https://lampo.azurewebsites.net/pubs/time-clocks.pdf>
- <https://homes.cs.washington.edu/~arvind/cs425/doc/mattern89virtual.pdf>

This ends the proposal section; below there are just some details that I wanted to keep track of!

Networking and deployment

The deployment uses a private ProxMox virtual network with static addressing so nodes can reliably discover each other. Traffic is logically separated between control-plane services (metadata, broker, and monitoring) and worker traffic (sync/job execution), and only required ports are opened to keep the environment realistic secure from the outside world.

- ProxMox bridge-based private network, static IP's
- Health checks for workers and services to support failure handling in demos
- Basic firewall rules to limit exposure and reduce debugging noise

Evaluation plan

The evaluation focuses on correctness, performance, and resilience under controlled fault injection.

File sync measurements

- Convergence time after bursts of edits
- Conflict rate under concurrent modification
- Bandwidth per replication batch
- Correctness: replica of hash equality and audit pass rates

Job queue measurements

- Throughput and end to end latency
- Retry correctness under induced worker crashes
- Duplicated execution rate (dedupe)

Resilience demonstrations

- Crash a worker vm mid-processing and show jobs complete somewhere else
- Restart services and show recovery without manual intervention

Expected challenges and mitigations

Distributed systems become difficult mainly around concurrency, failure, and operational edge cases. This project scopes “hard parts” carefully to ensure a successful implementation and a defensible report.

- **Conflicts in file sync:** rely on strong detection and preserve both versions; resolve via manual merge queue rather than risky automatic merges
- **Exactly once semantics:** implement at-least-once safely with idempotency keys + dedupe; document constraints and only attempt exactly-once where feasible
- **Partitions and message storms:** use bounded queues, rate limits, exponential backoff, and scheduled audits
- **Unexpected infra/hardware issues:** ProxMox snapshots, reproducible VM templates, and a written recovery runbook so the system can be rebuilt quickly

Deliverables

The final submission will include code, documentation, and repeatable demo steps that allow the system to be deployed and tested consistently.

- Code repository with **README** (setup, run, test, demo scripts)
- Interim report with draft abstract/introduction/design and partial implementation evidence
- Final report with architecture, implementation details, experiments, results/analysis, diagrams, future work, references
- Live demo showing scaling, fault injection, recovery, and measured performance metrics